



## **Trabalho Prático 1 - GRAFOS**

### **Análise de Rede Social**

INGRID LUÍZA RODRIGUES SILVA [5938]  
OTÁVIO FRANCISCO SABINO TAVARES [5912]  
VINÍCIUS ARRUDA MELO MIRANDA [5930]

## Sumário

|                                         |           |
|-----------------------------------------|-----------|
| <b>1. Introdução</b>                    | <b>3</b>  |
| <b>2. Metodologia</b>                   | <b>3</b>  |
| <b>3. Resultados do Desenvolvimento</b> | <b>4</b>  |
| 3.1 Estrutura e Representação do Grafo  | 4         |
| 3.2 Busca em largura                    | 5         |
| 3.3 Dijkstra                            | 6         |
| 3.4 Testes                              | 8         |
| <b>4. Compilação e Execução</b>         | <b>11</b> |
| <b>5. Considerações Finais</b>          | <b>11</b> |

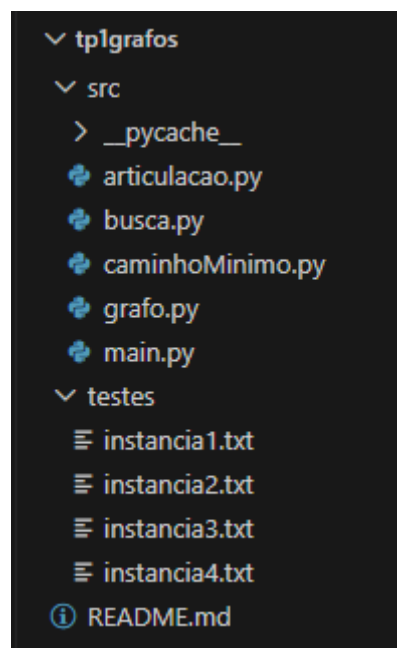
## 1. Introdução

Este trabalho da disciplina de Teoria e Modelo de Grafos busca desenvolver e praticar conceitos básicos de grafos a partir de um exemplo de uma rede social onde cada usuário pode estabelecer uma conexão com outro usuário, formando uma rede de relacionamentos. Para representar essa estrutura, são utilizados grafos não direcionados ponderados, nos quais cada vértice representa um usuário da rede social, cada aresta representa uma relação direta entre dois usuários, e o peso da aresta representa uma medida associada à relação entre os usuários.

O presente documento busca explicar pontos chave do código desenvolvido, além de apresentar passos básicos para utilizar a aplicação.

## 2. Metodologia

Para o desenvolvimento foi escolhida a linguagem de programação Python. A organização do repositório do projeto (Figura 1) foi feita considerando a pasta src, com os códigos, e a pasta testes, com os arquivos de texto a serem usados para executar as funcionalidades.



**Figura 1 - Organização do projeto.**

Foi utilizado o menu (Figura 2) para que o usuário especifique que tipo de funcionalidades vão ser testadas na execução. Cada funcionalidade implementa diferentes funções e classes a serem especificadas.

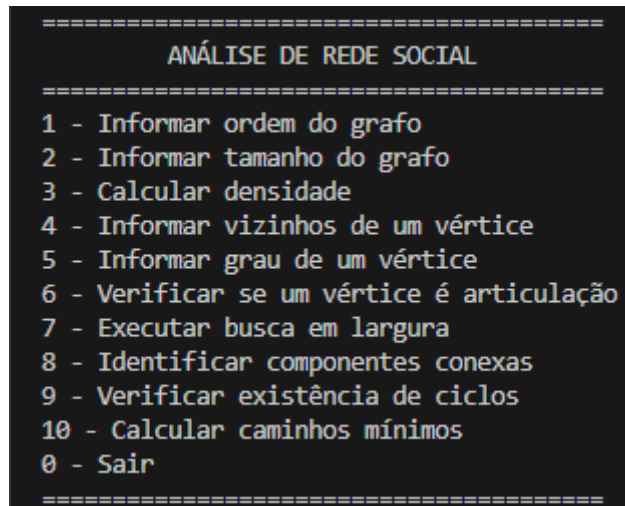


Figura 2 - Menu.

### 3. Resultados do Desenvolvimento

#### 3.1 Estrutura e Representação do Grafo

Nesta etapa do desenvolvimento, foi implementada a estrutura principal do grafo utilizando uma matriz de valores, responsável por armazenar as relações entre os vértices e os pesos das arestas. A classe Grafo realiza a leitura do arquivo de entrada, identifica a quantidade de vértices e constrói a matriz de adjacência, considerando que o grafo é não direcionado. Dessa forma, quando existe uma aresta entre dois vértices, o peso é armazenado nas duas posições correspondentes da matriz.

Também foram desenvolvidas as funções responsáveis por obter a ordem e o tamanho do grafo, calcular sua densidade, identificar os vizinhos de um determinado vértice e informar o seu grau. A função de tamanho contabiliza cada aresta apenas uma vez, enquanto a densidade é calculada considerando a quantidade de arestas em relação ao número máximo de arestas possíveis no grafo.

Além disso, foi implementada a identificação de vértices de articulação. Para isso, foi utilizado um algoritmo baseado em busca em profundidade (DFS), que mantém informações sobre a ordem em que os vértices são visitados e sobre o menor nível que pode ser alcançado a partir de cada vértice. Essas informações são utilizadas para verificar se a remoção de determinado vértice pode separar uma parte do grafo das demais.

```

1 def dfs(grafo, u, visitados, pai, profundidade, low, articulacoes, tempo):
2
3     visitados[u] = True
4
5     profundidade[u] = tempo[0]
6     low[u] = tempo[0]
7
8     tempo[0] += 1
9
10    filhos = 0
11
12    for v in range(grafo.ordem()):
13
14        if grafo.matriz[u][v] == 0:
15            continue
16
17        if not visitados[v]:
18
19            pai[v] = u
20            filhos += 1
21
22            dfs(
23                grafo,
24                v,
25                visitados,
26                pai,
27                profundidade,
28                low,
29                articulacoes,
30                tempo
31            )
32
33            low[u] = min(low[u], low[v])
34
35            if pai[u] == -1 and filhos > 1:
36                articulacoes.add(u)
37
38            if pai[u] != -1 and low[v] >= profundidade[u]:
39                articulacoes.add(u)
40
41        elif v != pai[u]:
42
43            low[u] = min(low[u], profundidade[v])

```

**Figura 3 - Função DFS.**

Para verificar o funcionamento das implementações, foram realizados testes utilizando arquivos de entrada contendo diferentes estruturas de grafos. Foram verificados os valores de ordem, tamanho e densidade, além dos vizinhos e graus de diferentes vértices. Também foram realizados testes específicos para verificar a identificação de vértices de articulação, incluindo grafos nos quais existem e não existem articulações.

### 3.2 Busca em largura

Esta etapa realiza uma travessia em largura (BFS) a partir de um vértice de origem específico. O algoritmo utiliza uma fila deque para processar os vértices em camadas de proximidade (primeiro os vizinhos diretos, depois os vizinhos dos vizinhos e assim por diante). Durante o percurso, a função diferencia as arestas que formaram a árvore de busca das demais arestas do grafo (arestas de retorno).

**Parâmetros:** O objeto grafo (que precisa fornecer o método `ordem()`) e o atributo `matriz` e o vértice início (indexado a partir de 1).

**Retorno:** Retorna uma tupla contendo duas listas. A primeira lista traz a ordem em que os vértices foram visitados a partir do nó inicial. A segunda lista contém as

arestas que não pertencem à árvore gerada pela BFS (representadas no formato de tuplas base-1). Caso o nó inicial seja inválido, retorna duas listas vazias.

A função de componentes conexas identifica e agrupa todos os subgrafos desconexos do grafo. A função percorre todos os vértices da estrutura e, sempre que encontra um vértice ainda não visitado em buscas anteriores, dispara um novo ciclo de BFS para mapear toda a sub-rede alcançável a partir dele.

**Parâmetros:** O objeto grafo.

**Retorno:** Retorna uma lista de listas, em que cada sublista representa uma componente conexa com os índices de seus vértices identificados a partir de 1.

Foi feita também a função que verifica a presença de ciclos em um grafo não orientado através da navegação por largura. O algoritmo percorre todas as componentes do grafo armazenando a relação de parentesco, com tuplas no formato (atual, pai) na fila. Se durante a expansão dos vizinhos a função encontrar um vértice que já foi visitado e esse vértice não for o pai direto do nó atual, confirma-se a existência de um caminho alternativo até ele, caracterizando um ciclo.

**Parâmetros:** O objeto grafo.

**Retorno:** Retorna um valor booleano: True se o grafo contiver ao menos um ciclo, ou False caso contrário (caracterizando uma floresta ou árvore).

### 3.3 Dijkstra

Para calcular caminhos mínimos e menores distâncias, foi utilizado o algoritmo de Dijkstra (Figura X), que calcula o caminho mínimo e a menor distância acumulada de um dado nó de origem até todos os outros nós do grafo.

**Estratégia de indexação:** O parâmetro início e as chaves de retorno utilizam índices baseados em 1. Internamente, o algoritmo ajusta esses valores para índices baseados em 0 para compatibilidade com matrizes e vetores na linguagem, cujos índices começam em 0.

```
def dijkstra(grafo, inicio):
    n = grafo.ordem()
    inicioIdx = inicio - 1

    distancias = [float('inf')] * n
    distancias[inicioIdx] = 0.0

    antecessores = [-1] * n

    pq = [(0.0, inicioIdx)]
```

Figura 4 - Inicializações.

**Representação e parâmetros:** Baseado na estrutura de matriz de adjacência, onde um valor  $> 0$  indica a existência de uma aresta e seu peso. Os parâmetros são: um grafo, e o nó de origem para o qual as distâncias serão calculadas para todos os vértices.

```
while pq:
    distAtual, u = heapq.heappop(pq)

    if distAtual > distancias[u]:
        continue

    for v in range(n):
        peso = grafo.matriz[u][v]
        if peso > 0:
            if distancias[u] + peso < distancias[v]:
                distancias[v] = distancias[u] + peso
                antecessores[v] = u
                heapq.heappush(pq, (distancias[v], v))
```

Figura 5 - Controle das distâncias.

**Retorno:** retorna uma tupla (distDict, caminhos): o primeiro item é o dicionário {verteice: distancia\_minima}. Se o nó for inalcançável, a distância será inf; o segundo item é o dicionário com os caminhos {verteice: [caminho\_percorrido]}, ou seja, a lista com a sequência de vértices do início ao destino.

```
caminhos = {}
for i in range(n):
    if distancias[i] == float('inf'):
        caminhos[i + 1] = []
    else:
        caminho = []
        atual = i
        while atual != -1:
            caminho.append(atual + 1)
            atual = antecessores[atual]
        caminho.reverse()
        caminhos[i + 1] = caminho

distDict = {i + 1: distancias[i] for i in range(n)}

return distDict, caminhos
```

**Figura 6 - Reconstrução dos caminhos.**

### 3.4 Testes

Foram feitas instâncias diferentes de teste.

**Grafo pequeno e conexo:** foi usado o teste “instancia1.txt” (Figura x), com 10 vértices, e obtém-se, por exemplo o resultado dos vizinhos do vértice 3 conforme Figura x).

```
10
1 2 2.5
1 4 4.0
2 3 1.5
2 5 3.2
3 4 2.0
4 6 1.8
5 6 2.7
6 4 0.4
7 3 3.4
8 7 1.3
9 6 2.0
10 5 7.0
10 1 5.9
```

**Figura 7 - Teste com 10 vértices.**

```
=====
ANÁLISE DE REDE SOCIAL
=====
1 - Informar ordem do grafo
2 - Informar tamanho do grafo
3 - Calcular densidade
4 - Informar vizinhos de um vértice
5 - Informar grau de um vértice
6 - Verificar se um vértice é articulação
7 - Executar busca em largura
8 - Identificar componentes conexas
9 - Verificar existência de ciclos
10 - Calcular caminhos mínimos
0 - Sair
=====
Digite a opção: 4
Digite o vértice: 3
Vizinhos: [2, 4, 7]
```

**Figura 8 - Resultado de vizinhos do vértice 3.**

**Grafo desconexo:** foi usada a instância 2 (Figura x) para este teste, com vértices desconexos (o vértice 4 é inalcançável), e foi realizada, por exemplo a escolha de caminhos mínimos (Figura x).

|   |   |     |
|---|---|-----|
| 4 |   |     |
| 1 | 2 | 1.0 |
| 2 | 3 | 1.0 |

Figura 9 - Instância 2.

```

Digite o nome do arquivo do grafo: instancia2.txt
=====
ANÁLISE DE REDE SOCIAL
=====
1 - Informar ordem do grafo
2 - Informar tamanho do grafo
3 - Calcular densidade
4 - Informar vizinhos de um vértice
5 - Informar grau de um vértice
6 - Verificar se um vértice é articulação
7 - Executar busca em largura
8 - Identificar componentes conexas
9 - Verificar existência de ciclos
10 - Calcular caminhos mínimos
0 - Sair
=====
Digite a opção: 10
Digite o vertice de origem: 1
para o vertice 1 : distancia = 0.0 | caminho = [1]
para o vertice 2 : distancia = 1.0 | caminho = [1, 2]
para o vertice 3 : distancia = 2.0 | caminho = [1, 2, 3]
para o vertice 4 : distancia = Inalcançável | caminho = []

```

Figura 10 - Resultado de caminhos mínimos.

**Grafo com baixa densidade:** foi usada a instância 3 (Figura x), com 55 vértices, mas apenas 6 arestas, e foi testado, por exemplo, a análise de densidade (Figura x).

|    |    |     |
|----|----|-----|
| 55 |    |     |
| 1  | 2  | 1.0 |
| 2  | 3  | 1.0 |
| 6  | 7  | 3.0 |
| 15 | 1  | 5.0 |
| 20 | 31 | 9.0 |
| 31 | 45 | 1.0 |

Figura 11 - Instância 3.

```
=====
ANÁLISE DE REDE SOCIAL
=====
1 - Informar ordem do grafo
2 - Informar tamanho do grafo
3 - Calcular densidade
4 - Informar vizinhos de um vértice
5 - Informar grau de um vértice
6 - Verificar se um vértice é articulação
7 - Executar busca em largura
8 - Identificar componentes conexas
9 - Verificar existência de ciclos
10 - Calcular caminhos mínimos
0 - Sair
=====
Digite a opção: 3
Densidade do grafo: 0.00404040404040404
```

Figura 12 - Resultado da densidade.

**Grafo maior:** foi feita uma instância com 580 vértices e mais de 2000 arestas (instanci4.txt na pasta testes). É possível obter, por exemplo, o tamanho do grafo (Figura x).

```
=====
ANÁLISE DE REDE SOCIAL
=====
1 - Informar ordem do grafo
2 - Informar tamanho do grafo
3 - Calcular densidade
4 - Informar vizinhos de um vértice
5 - Informar grau de um vértice
6 - Verificar se um vértice é articulação
7 - Executar busca em largura
8 - Identificar componentes conexas
9 - Verificar existência de ciclos
10 - Calcular caminhos mínimos
0 - Sair
=====
Digite a opção: 2
Tamanho do grafo: 2494
```

Figura 13 - Resultado de tamanho para grafo maior.

## **4. Compilação e Execução**

Para executar os códigos pode ser usado no Windows o comando de terminal “python src/main.py”, e no Linux pode ser usado o comando “python3 src/main.py”.

## **5. Considerações Finais**

O desenvolvimento deste Trabalho Prático permitiu consolidar a aplicação teórica da Teoria dos Grafos em um cenário prático e intuitivo, a modelagem e análise de uma Rede Social, na qual os vértices representam os usuários e as arestas traduzem as conexões entre eles. A partir das funcionalidades implementadas, foi possível extrair métricas fundamentais para compreender o comportamento e a estrutura dessa rede: métricas estruturais (ordem, tamanho e densidade); vizinhos e grau; articulação; busca em largura e componentes conexas; ciclos e caminhos mínimos.

Em suma, a implementação dos algoritmos demonstrou que a Teoria dos Grafos oferece as ferramentas para transformar dados brutos em informações úteis.